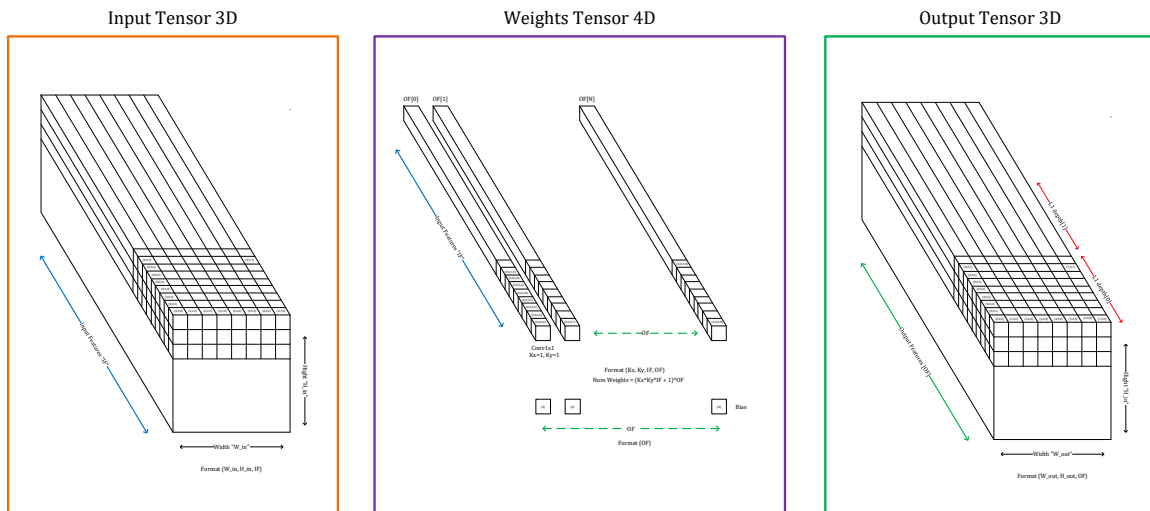


HW1 - Speeding up a 1×1 Convolution by Changing the Data Layout



Approach

We implemented a 1×1 convolution (pointwise convolution), a common CNN layer. For each pixel it is just a weighted sum across the input channels:

$$\text{out}[co][h][w] = \sum_{ci} \text{in}[ci][h][w] * \text{weight}[co][ci]$$

The amount of arithmetic is fixed, so any speedup has to come from how the data is arranged in memory and not from doing less work. That makes it a good fit for showing the connection between the code and the hardware.

Both versions use the same deterministic inputs and print the same checksum, so we confirm the outputs are identical with `diff`. Both are compiled with the same flags (`-O3 -mavx2 -mfma`), so the difference comes from our change and not the compiler. We used $C_{in} = C_{out} = 512$ and $H = W = 128$; each tensor is about 32 MB, larger than the L3 cache. With small tensors that fit in cache the two versions ran almost the same (about 60 ms vs 58 ms), so the effect only shows up once the data spills out of cache.

Profiling used `perf` on a virtualized Ubuntu host, where the cycles, LLC, and branch-misses counters were unavailable. The L1 data-cache counters worked, and since our argument is about L1 locality those are the ones that matter; we use wall-clock time and instructions per second in place of the cycle counter.

> **Note on layout.** NCHW and NHWC name the order the tensor dimensions are kept in memory, > from slowest- to fastest-changing: N = batch, C = channels, H = height, W = width. In NCHW >

width changes fastest, so one channel image is contiguous but the next channel of the > same pixel is H×W elements away. In NHWC the channels change fastest, so all channels of a > pixel sit next to each other. Our two versions differ only in this choice.

Unoptimized version (conv1x1_naive.cpp)

Standard NCHW layout, loops (co, h, w, ci). The inner loop sums over ci, but in NCHW two channels of the same pixel are H×W = 16384 floats apart, which is 64 KB:

```
for (int ci = 0; ci < Cin; ++ci)
    acc += in[ci*HW + h*W + w] * wt[co*Cin + ci]; // in[] jumps 64 KB each step
```

Every step lands on a new cache line and uses only 4 bytes of it before jumping 64 KB away, so the line is wasted. The prefetcher expects small steady strides, not 64 KB jumps, so it cannot help. The input is also re-read once per output channel.

Metric	Value
Runtime	11.37 s
Instructions	4.459×10^9
L1-dcache loads	1.650×10^9
L1-dcache load misses	1.587×10^9 (96.17 % miss rate)
Instructions per second	about 0.38 billion

The 96 % miss rate is the key number: nearly every load misses L1 and waits on memory, so the CPU mostly sits idle and retires only about 0.38 billion instructions per second.

Optimized version (conv1x1_opt.cpp)

Two changes, same math and same output:

1. **NCHW → NHWC**. After transposing the input, the channels of one pixel are adjacent, so the reduction over ci walks straight through memory and the compiler can vectorize it with FMA. The weight row is already contiguous. 2. **Register blocking over 4 output channels**. Each input value is loaded once and reused for four output channels held in registers.

```
const float* xin = &in_nhwc[p*Cin]; // Cin contiguous floats
for (int co = 0; co < Cout; co += 4) {
    float a0=0, a1=0, a2=0, a3=0;
    for (int ci = 0; ci < Cin; ++ci) {
        float x = xin[ci];
        a0 += x*w0[ci]; a1 += x*w1[ci];
        a2 += x*w2[ci]; a3 += x*w3[ci]; // x reused 4 times
    }
}
```

Metric	Value
Runtime	2.28 s
Instructions	8.763×10^9
L1-dcache loads	5.404×10^9
L1-dcache load misses	0.295×10^9 (5.45 % miss rate)

Instructions per second	about 3.67 billion
-------------------------	--------------------

The miss rate drops from 96 % to about 5 % and runtime falls roughly 5×. The surprising part is that the fast version runs more instructions and does more loads, yet still wins, because those extra loads almost all hit L1 and are nearly free.

Comparison

Metric	Naive	Optimized	Change
Runtime	11.37 s	2.28 s	about 5× faster
L1 miss rate	96.17 %	5.45 %	much lower
L1 load misses	1.587×10^9	0.295×10^9	~5.4× fewer
L1 loads	1.650×10^9	5.404×10^9	~3.3× more
Instructions	4.459×10^9	8.763×10^9	~2× more
Instructions/sec	0.38 B	3.67 B	~9.7× higher

Both versions do the same 4.3 billion multiply-adds, so the difference is not about doing less work. The naive version stalls because nearly every load misses L1 and waits on a distant cache line it barely uses. The optimized version streams the same data sequentially, so about 95 % of loads hit L1. The point is that what costs time is not how many loads you do but how long each one takes: an L1 hit is a few cycles while a miss is tens to hundreds, so trading fewer missing loads for more hitting loads is a big net win. The instruction rate rising about 9.7× shows the CPU is finally kept busy instead of waiting on memory.

Things we tried

- Turning on compiler optimization alone did not help the naive version, since the compiler cannot reorder the memory accesses. That is why we kept the flags identical for both.
- Small tensor sizes fit in cache and showed almost no difference; we needed a size that spills out of cache.
- Full GEMM-style tiling would be faster but obscures the single clear idea (layout plus reuse), so we left it out.

How to reproduce

`bash run.sh` compiles both versions, checks that their outputs match, and runs `perf`. If the machine with `perf` has no compiler, build with `build_portable.sh` elsewhere and run `perf.sh` on the copied `bin/`. All progress prints are behind a `VERBOSE` flag (`-DVERBOSE=0`) so the profiling runs stay clean.